Product Requirements Document (PRD)

Product Name: Global South Index Directory (GSID)

Document Version: 1.1

Target Platform: Python / Multi-Agent System (via Google Antigravity)

1. Product Vision & Overview

The Global South Index Directory (GSID) is a multi-agent data extraction system designed to build a comprehensive, accessible database of researchers from Africa, Asia, and Latin America. By prioritizing Open Scholarly APIs (like OpenAlex and Zenodo) and ethically scraping open-access platforms (like AJOL and DOAJ), the system will bypass hostile "walled gardens" to sustainably harvest names, affiliations, and email addresses.

The ultimate goal is to improve the global visibility of Global South research, powering large-scale indexing initiatives, collaboration networks, and conference outreach.

2. Target Audience & Use Cases

- **Primary User:** Non-technical Project Manager (interacting via Google Antigravity).
- **End-Users of Data:** Research institutions, open-access indexers, grant providers, and academic conference organizers seeking to diversify their networks.
- **Core Use Case:** The user inputs target regions (e.g., "Africa", "Latin America") and academic disciplines, and the system autonomously queries APIs and scrapes designated sites to produce a clean, deduplicated CSV of researcher contacts.

3. Scope & Data Sources

To ensure system resilience and compliance with Terms of Service, data sources are strictly categorized:

3.1. In-Scope: Primary Data Sources (API-First)

- **OpenAlex API:** The primary engine for discovering affiliated authors.
- **DOAJ (Directory of Open Access Journals) API:** For metadata on open-access publications.
- **Zenodo API:** For researchers publishing open datasets and preprints.

3.2. In-Scope: Secondary Data Sources (Ethical Web Scraping)

- **AJOL (African Journals OnLine):** Targeted scraping for author details using BeautifulSoup/Selenium.
- **NJOL (Nigerian Journals OnLine):** Targeted scraping.

3.3. Out-of-Scope (Do Not Target)

- Scopus, Web of Science, ResearchGate, Academia.edu, Taylor & Francis. (Attempting to scrape these will result in IP bans and system breakage; their data will instead be inferred via OpenAlex where possible).

4. System Architecture: The Multi-Agent Design

The system will be built using a modular, multi-agent architecture managed by Google Antigravity.

- **Agent 1: The API Harvester:** Specialized in interacting with REST APIs (OpenAlex, Zenodo). Handles pagination, API keys (if necessary), and JSON parsing.
- **Agent 2: The Ethical Scraper:** Specialized in parsing HTML from AJOL/NJOL. Includes built-in rate-limiting (e.g., 2-second delays between page requests) to prevent server overload.
- **Agent 3: The Data Synthesizer:** Takes raw data from Agents 1 and 2, standardizes the formatting (Name, Email, Institution, Region, Source), deduplicates records based on email addresses, and exports to CSV.

5. Functional Requirements (User Stories)

- **FR1:** As a user, I want the system to accept a list of target countries/regions so that it only fetches researchers affiliated with those locations.
- **FR2:** As a user, I want the API Harvester to query OpenAlex for authors from the specified regions and extract display_name, last_known_institution, and email (if available in the metadata).
- **FR3:** As a user, I want the Ethical Scraper to navigate AJOL/NJOL article pages to extract corresponding author emails using regular expressions (Regex) designed for email structures.
- **FR4:** As a user, I want the Data Synthesizer to automatically merge data from all sources into a single, standardized Pandas DataFrame.
- **FR5:** As a user, I want the system to output a finalized .csv file named gsid_export_[DATE].csv.

6. Non-Functional Requirements

- **Reliability:** The system must handle failed requests gracefully (e.g., if a webpage times out, it logs the error and moves to the next URL without crashing the whole script).
- **Ethical Compliance:** Web scraping scripts *must* respect robots.txt and implement randomized sleep intervals between 1 and 5 seconds per request.
- **Data Privacy:** The system will only extract publicly available contact information intended for academic correspondence.
- **Maintainability:** Code generated by Antigravity must be heavily commented and modular so the non-technical PM can easily read the logic flow.

7. Success Metrics

- **Yield:** Extraction of at least 1,000 unique, valid researcher profiles per region in the initial test run.
- **Accuracy:** A less than 5% bounce rate on the extracted email addresses.
- **Stability:** The system runs from start to CSV export without crashing or requiring manual code intervention.

8. Antigravity Implementation Plan (For the PM)

To build this, feed the following phases into Google Antigravity sequentially:

1. **Phase 1 (Setup):** Upload this PRD to the workspace to establish context.
2. **Phase 2 (API Build):** Instruct Antigravity: *"Build Agent 1: The API Harvester based on the PRD. Write a Python script to query the OpenAlex API for authors in Nigeria, Brazil, and India. Output a sample CSV of 50 results."*
3. **Phase 3 (Scraper Build):** Instruct Antigravity: *"Build Agent 2: The Ethical Scraper. Write a Python script using BeautifulSoup to scrape the latest issue of an AJOL journal, extracting author names and emails. Implement a 3-second sleep delay."*
4. **Phase 4 (Integration):** Instruct Antigravity: *"Build Agent 3: The Data Synthesizer. Write a script to take the outputs from Phase 2 and Phase 3, deduplicate them, and generate the final CSV."*